

Find
■ Case-sensitive ■ Regexp search
Path

Showing [eb8ceeb](#) from 2026-09-27: Revert "Bug [2060428](#) - Part 10 - Use the new Python Cargo actions by default r=firefox-build-system-reviewers,sergesanspaille" for causing fuzzing ccov bustage at cargo actions

[firefox-main](#) — [/mobile/android/geckoview/src/main/java/org/mozilla/geckoview](#)

Navigation			
Name	Description	Size	Coverage
AIFeaturesController.java	Coordinates messaging between GeckoView and the AI feature toolkit. <p>Marked as experimental due to active development.	10726	:
AllowOrDeny.java	This represents a decision to allow or deny a request.	450	:
Autocomplete.java	The Autocomplete API provides a way to leverage Gecko's input form handling for autocompletion. <p>The API is split into two parts: 1. Storage-level delegates. 2. User-prompt delegates. <p>The storage-level delegates connect Gecko mechanics to the app's storage, e.g., retrieving and storing of login entries. <p>The user-prompt delegates propagate decisions to the app that could require user choice, e.g., saving or updating of login entries or the selection of a login entry out of multiple options. <p>Throughout the documentation, we will refer to the filling out of input forms using two terms: 1. Autofill: automatic filling without user interaction. 2. Autocomplete: semi-automatic filling that requires user prompting for the selection. <h2>Examples</h2><h3>Autocomplete/Fetch API</h3> <p>GeckoView loads <code>https://example.com</code> which contains (for the purpose of this example) elements resembling a login form, e.g., <pre><code>&lt;form&gt; &lt;input type=&quot;text&quot; placeholder=&quot;username&quot;&gt; &lt;input type=&quot;password&quot; placeholder=&quot;password&quot;&gt; &lt;input type=&quot;submit&quot; value=&quot;submit&quot;&gt; &lt;/form&gt; </code></pre> <p>With the document parsed and the login input fields identified, GeckoView dispatches a <code>StorageDelegate.onLoginFetch(&quot;example.com&quot;)</code> request to fetch logins for the given domain. <p>Based on the provided login entries, GeckoView will attempt to autofill the login input fields, if there is only one suitable login entry option. <p>In the case of multiple valid login entry options, GeckoView dispatches a <code>GeckoSession.PromptDelegate.onLoginSelect</code> request, which allows for user-choice delegation. <p>Based on the returned login entries, GeckoView will attempt to autofill/autocomplete the login input fields. <h3>Update API</h3> <p>When the user submits some login input fields, GeckoView dispatches another <code>StorageDelegate.onLoginFetch(&quot;example.com&quot;)</code> request to check whether the submitted login exists or whether it's a new or updated login entry. <p>If the submitted login is already contained as-is in the collection returned by <code>onLoginFetch</code>, then GeckoView dispatches <code>StorageDelegate.onLoginUsed</code> with the submitted login entry. <p>If the submitted login is a new or updated entry, GeckoView dispatches a sequence of requests to save/update the login entry, see the Save API example. <h3>Save API</h3> <p>The user enters new or updated (password) login credentials in some login input fields and submits explicitly (submit action) or by navigation. GeckoView identifies the entered credentials and dispatches a <code>GeckoSession.PromptDelegate.onLoginSave(session, request)</code> with the provided credentials. <p>The app may dismiss the prompt request via <code>return GeckoResult.fromValue(prompt.dismiss())</code> which terminates this saving request, or confirm it via <code>return GeckoResult.fromValue(prompt.confirm(login))</code> where	60783	:

Name	Description	Size	Coverage
	<code><code>login </code></code> either holds the credentials originally provided by the prompt request (<code><code> prompt.logins[0]</code></code>) or a new or modified login entry. <p>The login entry returned in a confirmed save prompt is used to request for saving in the runtime delegate via <code><code>StorageDelegate.onLoginSave(login)</code></code>. If the app has already stored the entry during the prompt request handling, it may ignore this storage saving request.
 @see GeckoRuntime#setAutocompleteStorageDelegate
 @see GeckoSession#setPromptDelegate
 @see GeckoSession.PromptDelegate#onLoginSave
 @see GeckoSession.PromptDelegate#onLoginSelect		
Autofill.java	Autofill functionality for forms and input fields.	42409	-
Base64Utils.java	This class exposes the Base64 URL encode/decode functions from Gecko. They are different from android.util.Base64 in that they always use URL encoding, no padding, and are constant time. The last bit is important when dealing with values that might be secret as we do with Web Push.	766	-
BasicSelectionActionDelegate.java	Class that implements a basic SelectionActionDelegate. This class is used by GeckoView by default if the consumer does not explicitly set a SelectionActionDelegate. <p>To provide custom actions, extend this class and override the following methods, <p>1) Override {@link #getAllActions} to include custom action IDs in the returned array. This array must include all actions, available or not, and must not change over the class lifetime. <p>2) Override {@link #isActionAvailable} to return whether a custom action is currently available. <p>3) Override {@link #prepareAction} to set custom title and/or icon for a custom action. <p>4) Override {@link #performAction} to perform a custom action when used.	22298	-
CompositorController.java	Controller for managing compositor-related functionality.	3749	-
ContentBlocking.java	Content Blocking API to hold and control anti-tracking, cookie and Safe Browsing settings.	89693	-
ContentBlockingController.java	ContentBlockingController is used to manage and modify the content blocking exception list. This list is shared across all sessions.	14178	-
ContentInputStream.java	This class provides an {@link OutputStream} wrapper for a Gecko nsIOutputStream (or really, nsIRequest).	6289	-
CrashHandler.java	Handler for processing application crashes and sending crash reports. The CrashPullController offers a way to get notified of JS level Crash Pull events, that may be triggered by incoming Remote Settings requests offering developpers a way to collect specific crash reports that may be required for investigation on specific bugs. <p>Upon the event triggered by the JS level code in GeckoViewStartup and assuming a CrashPullController.Delegate was registered, then its onCrashPull gets called to perform the rest of the handling, i.e., notifying the user to ask for those crash submission.	19977	-
CrashPullController.java	Sends a crash report to the Mozilla Socorro crash report server.	3531	-
CrashReporter.java	Additional metadata about a deprecation notice.	15802	-
DeprecationSchedule.java	GeckoView APIs annotated with {@link ExperimentalGeckoViewApi} are subject to change or else inherently more dangerous or unstable. <p>This annotation covers: API signatures that may change without notice or deprecation period. API behavior and interactions that may change without notice. Features that are under heavy development and are considered unstable. Features that have inherent risk or potentially destabilize other parts of GeckoView. <p>Only use an API annotated with this if you fully understand the API and accept the risks.	1331	-
doc-files			-
ExperimentalGeckoViewApi.java	This delegate is used to pass experiment information between the embedding application and GeckoView. <p>An experiment is used to	1472	-
ExperimentDelegate.java		7417	-

Name	Description	Size	Coverage
GeckoDisplay.java	give users different application behavior in order to learn and improve upon what features users prefer the most. This is accomplished by providing users different application experiences and collecting data about how the differing experiences impacted user behavior. Applications use a GeckoDisplay instance to provide {@link GeckoSession} with a {@link Surface} for displaying content. To ensure drawing only happens on a valid {@link Surface}, {@link GeckoSession} will only use the provided {@link Surface} after {@link #surfaceChanged(SurfaceInfo)} is called and before {@link #surfaceDestroyed()} returns.	25593	-
GeckoEditable.java	GeckoEditable implements only some functions of Editable The field mText contains the actual underlying SpannableStringBuilder/Editable that contains our text.	104139	-
GeckoFontScaleListener.java	A class that automatically adjusts font size settings for web content in Gecko in accordance with the device's OS font scale setting. @see android.provider.Settings.System#FONT_SCALE	4955	-
GeckoInputConnection.java	package	31549	-
GeckoInputStream.java	This class provides an {@link InputStream} wrapper for a Gecko nsIChannel (or really, nsIRequest).	5873	-
GeckoPreferenceController.java	Class is used to access and manipulate Gecko preferences through GeckoView. <p>This API is experimental because manipulating Gecko preferences is risky and can interfere with browser operation without special care.	28744	-
GeckoResult.java		38576	-
GeckoRuntime.java	Runtime environment for Gecko-based applications.	46599	-
GeckoRuntimeSettings.java	Settings for configuring the Gecko runtime environment.	94676	-
GeckoSession.java	A session for interacting with web content in Gecko. <p>GeckoSession represents a single tab or browser window and provides APIs for loading content, handling navigation, managing permissions, and interacting with web content through various delegate interfaces.	319677	-
GeckoSessionHandler.java	package	2977	-
GeckoSessionSettings.java	Configuration settings for a GeckoSession.	23892	-
GeckoView.java	A view container that hosts Gecko rendering, manages its surface, and dispatches input/events.	46972	-
GeckoViewInputStream.java	This class provides a Gecko nsIInputStream wrapper for a Java {@link InputStream} .	4174	-
GeckoViewPrintDocumentAdapter.java	Print document adapter for GeckoView content.	8630	-
GeckoVRManager.java	Interface for registering the external VR context with WebVR. The context must be registered before Gecko is started. This API is not intended for external consumption. To see an example of how it is used please see the https://github.com/MozillaReality/FirefoxReality Firefox Reality browser. @see External VR Context GeckoWebExecutor is responsible for fetching a {@link WebRequest}, and delivering a {@link WebResponse} to the caller via {@link #fetch(WebRequest)}. Example: <pre>final GeckoWebExecutor executor = new GeckoWebExecutor(); final GeckoResult<WebResponse> result = executor.fetch(new WebRequest.Builder("https://example.org/json").header("Accept", "application/json").build()); result.then(response -&gt; { // Do something with response });</pre>	1446	-
GeckoWebExecutor.java		7036	-
HandlerThread.java	Indicates that the launching thread must have an Android {@link android.os.Handler} to execute due to the {@link GeckoResult} . <p>If called from a thread without the required Handler, the code will throw	1074	-

Name	Description	Size	Coverage
	{@link IllegalStateException} with message "Must have a Handler". @see GeckoResult		
Image.java	Represents an Web API image resource as used in web app manifests and media session metadata.	1834	=
IPProtectionController.java	Controller for managing IP protection state.	31547	=
MediaSession.java	The MediaSession API provides media controls and events for a GeckoSession. This includes support for the DOM Media Session API and regular HTML media content. @see US/docs/Web/API/MediaSession >Media Session API	25384	=
OrientationController.java	Controller for managing device orientation changes.	1771	=
OverscrollEdgeEffect.java	Edge effect implementation for overscroll behavior.	7011	=
	This package contains the public interfaces for the library. {@link org.mozilla.geckoview.GeckoRuntime} is the entry point for starting and initializing Gecko. You can use this to preload Gecko before you need to load a page or to configure features such as crash reporting. {@link org.mozilla.geckoview.GeckoSession} is where most interesting work happens, such as loading pages. It relies on {@link org.mozilla.geckoview.GeckoRuntime} to talk to Gecko. {@link org.mozilla.geckoview.GeckoView} is the embeddable {@link android.view.View}. This is the most common way of getting a {@link org.mozilla.geckoview.GeckoSession} onto the screen. Permissions This library does not request any dangerous permissions in the manifest, though it's possible that some web features may require them. For instance, WebRTC video calls would need the {@link android.Manifest.permission#CAMERA} and {@link android.Manifest.permission#RECORD_AUDIO} permissions. Declaring these are at the application's discretion. If you want full web functionality, the following permissions should be declared: {@link android.Manifest.permission#ACCESS_COARSE_LOCATION}. {@link android.Manifest.permission#ACCESS_FINE_LOCATION}. {@link android.Manifest.permission#READ_EXTERNAL_STORAGE} {@link android.Manifest.permission#WRITE_EXTERNAL_STORAGE} {@link android.Manifest.permission#CAMERA} {@link android.Manifest.permission#RECORD_AUDIO} {@link android.Manifest.permission#ACCESS_LOCAL_NETWORK} For a detailed change log of the API see: CHANGELOG.	2046	=
PageExtractionController.java	Manages the page content extraction	9321	=
PanZoomController.java	Manages panning, zooming, and input event handling for a GeckoSession. This controller processes touch, mouse, and scroll events to update the web content viewport, supports fling and smooth scrolling behaviors, and reports input handling results.	35556	=
ParcelableUtils.java		508	=
PdfViewerController.java	The PDF viewer controller coordinates messaging between GeckoView and the PDF viewer when the session contains a PDF.	3988	=
ProfilerController.java	ProfilerController is used to manage GeckoProfiler related features. If you want to add a profiler marker to mark a point in time (without a duration) you can directly use <code>profilerController.addMarker("marker name")</code>. Or if you want to provide more information, you can use <code>profilerController.addMarker("marker name", "extra information")</code>. If you want to add a profiler marker with a duration (with start and end time) you can use it like this: <code>Double startTime = profilerController.getProfilerTime(); ...some code you want to measure... profilerController.addMarker("name", startTime);</code> Or you can capture start and end time in somewhere, then add the marker in	8075	=

Name	Description	Size	Coverage
	somewhere else: <code>Double startTime = profilerController.getProfilerTime(); ...some code you want to measure (or end time can be collected in a callback)... Double endTime = profilerController.getProfilerTime(); ...somewhere else in the codebase... profilerController.addMarker("name", startTime, endTime);</code> Here's an <code>addMarker</code> example with all the possible parameters: <code>Double startTime = profilerController.getProfilerTime(); ...some code you want to measure... Double endTime = profilerController.getProfilerTime(); ...somewhere else in the codebase... profilerController.addMarker("name", startTime, endTime, "extra information");</code> <code>isProfilerActive</code> method is handy when you want to get more information to add inside the marker, but you think it's going to be computationally heavy (and useless) when profiler is not running: <pre>Double startTime = profilerController.getProfilerTime(); ...some code you want to measure... if (profilerController.isProfilerActive()) { String info = aFunctionYouDoNotWantToCallWhenProfilerIsNotActive(); profilerController.addMarker("name", startTime, info); }</pre> FIXME(bug 1618560): Currently only works in the main thread.		
PromptController.java	package	31666	=
RuntimeSettings.java	Base class for (nested) runtime settings. <p>Handles pref-based settings. Please extend this class when adding nested settings for GeckoRuntimeSettings.	11255	=
ScreenLength.java	ScreenLength is a class that represents a length on the screen using different units. The default unit is a pixel. However lengths may be also represented by a dimension of the visual viewport or of the full scroll size of the root document.	5563	=
SessionAccessibility.java	Accessibility support for GeckoSession.	35627	=
SessionFinder.java	{@code SessionFinder} instances returned by {@link GeckoSession#getFinder()} performs find-in-page operations.	4674	=
SessionPdfFileSaver.java	{@code PdfFileSaver} instances returned by {@link GeckoSession#getPdfFileSaver()} performs save operation.	4032	=
SessionTextInput.java	{@code SessionTextInput} handles text input for {@code GeckoSession} through key events or input methods. It is typically used to implement certain methods in {@link android.view.View} such as {@link android.view.View#onCreateInputConnection}, by forwarding such calls to corresponding methods in {@code SessionTextInput}. <p>For full functionality, {@code SessionTextInput} requires a {@link android.view.View} to be set first through {@link #setView}. When a {@link android.view.View} is not set or set to null, {@code SessionTextInput} will operate in a reduced functionality mode. See {@link #onCreateInputConnection} and methods in {@link GeckoSession.TextInputDelegate} for changes in behavior in this viewless mode.	15681	=
SlowScriptResponse.java	Used by a ContentDelegate to indicate what action to take on a slow script event. @see GeckoSession.ContentDelegate#onSlowScript(GeckoSession,String)	584	=
StorageController.java	Manage runtime storage data. <p>Retrieve an instance via {@link GeckoRuntime#getStorageController}.	12619	=
TranslationsController.java	The translations controller coordinates the session and runtime messaging between GeckoView and the translations toolkit.	54033	=
WebAuthnTokenManager.java	package	31277	=
WebExtension.java	Represents a WebExtension that may be used by GeckoView.	114754	=
WebExtensionController.java	Controller for managing WebExtensions within a GeckoRuntime instance. Provides functionality for installing, uninstalling, enabling, disabling, and managing delegates for WebExtensions.	73357	=
WebMessage.java	This is an abstract base class for HTTP request and response types.	4097	=
WebNotification.java	This class represents a single Web Notification. These can be	10143	=

Name	Description	Size	Coverage
WebNotificationAction.java	received by connecting a {@link WebNotificationDelegate} to {@link GeckoRuntime} via {@link GeckoRuntime#setWebNotificationDelegate(WebNotificationDelegate)} . This class corresponds to <code>nsIAlertAction</code> in Gecko, which again largely corresponds to each member of Notification.actions . It's passed to {@link WebNotification} and can be retrieved from it.	2137	=
WebNotificationDelegate.java	Delegate for handling web notification events.	1088	=
WebPushController.java	Controller for managing web push notifications.	5192	=
WebPushDelegate.java	Delegate for handling web push subscription events.	2435	=
WebPushSubscription.java	This class represents a single Web Push subscription, as described in the Web Push API specification. This is a low-level interface, allowing applications to do all of the heavy lifting themselves. It is recommended that consumers have a thorough understanding of the Web Push API, especially RFC 8291. Only trivial sanity checks are performed on the values held here. The application must ensure it is generating compliant keys/secrets itself.	6571	=
WebRequest.java	<code>WebRequest</code> represents an HTTP[S] request. The typical pattern is to create instances of this class via {@link WebRequest.Builder} , and fetch responses via {@link GeckoWebExecutor#fetch(WebRequest)} .	7650	=
WebRequestError.java	<code>WebRequestError</code> is simply a container for error codes and categories used by {@link GeckoSession.NavigationDelegate#onLoadError(GeckoSession, String, WebRequestError)} .	13772	=
WebResponse.java	<code>WebResponse</code> represents an HTTP[S] response. It is normally created by {@link GeckoWebExecutor#fetch(WebRequest)} .	7854	=

This page was generated by [Searchfox](#): [source code repository](#) | [configuration repository](#) | [see open bugs](#) | [file a new bug](#) | [documents](#)